



Guía de Despliegue

App Metodología 5S → Vercel + Supabase

Paso a paso para desplegar tu aplicación 5S de forma permanente y gratuita en la nube

Next.js 16

Vercel

Supabase

PostgreSQL

Mayo 2026

Contenido

- 1 Resumen de cambios realizados
- 2 Requisitos previos (cuentas gratuitas)
- 3 Paso 1: Crear proyecto en Supabase
- 4 Paso 2: Configurar base de datos PostgreSQL
- 5 Paso 3: Crear bucket de almacenamiento
- 6 Paso 4: Subir código a GitHub
- 7 Paso 5: Desplegar en Vercel
- 8 Paso 6: Configurar variables de entorno
- 9 Paso 7: Ejecutar migración de base de datos
- 10 Flujo de trabajo diario
- 11 Solución de problemas

1. Resumen de cambios realizados

Tu app ha sido adaptada para funcionar en Vercel (servidor sin estado) con Supabase (base de datos PostgreSQL + almacenamiento de fotos en la nube). Estos son los cambios principales:

Componente	Antes (Z)	Ahora (Vercel)
Base de datos	SQLite (archivo local)	PostgreSQL (Supabase)
Almacenamiento fotos	Archivos en /public/	Supabase Storage (nube)
Hosting	Servidor temporal Z	Vercel (permanente)
API Progress	Rutas dinámicas anidadas	Query parameters
projectId	Parcialmente usado	Obligatorio en todas las APIs

Compatibilidad hacia atrás

La app funciona tanto en Z (desarrollo local) como en Vercel (producción). Si Supabase no está configurado, las fotos se guardan localmente como antes.

Archivos creados/modificados

- **Nuevo:** `src/lib/supabase-storage.ts` - Subida de fotos a Supabase Storage
- **Nuevo:** `src/app/api/progress/step/route.ts` - API de progreso con query params
- **Nuevo:** `.env.example` - Plantilla de variables de entorno
- **Modificado:** `prisma/schema.prisma` - SQLite a PostgreSQL con índices
- **Modificado:** `src/lib/db.ts` - Optimizado para serverless
- **Modificado:** `src/app/api/upload/route.ts` - Supabase Storage con fallback local
- **Modificado:** `next.config.ts` - Sin standalone, con dominios Supabase
- **Modificado:** Todos los componentes 5S - Pasan `projectId` a las APIs
- **Modificado:** Todas las APIs - Usan `projectId` correctamente

2. Requisitos previos (cuentas gratuitas)

Necesitas crear 3 cuentas gratuitas. No requieren tarjeta de crédito:

Servicio	URL	Para qué	Plan gratis
GitHub	github.com	Almacenar tu código	Ilimitado
Supabase	supabase.com	Base de datos + fotos	500MB DB + 1GB Storage
Vercel	vercel.com	Hosting de la app	Ilimitado (hobby)

Importante

Regístrate con tu cuenta de GitHub en Supabase y Vercel. Así será más fácil conectar todo después.

- ☐ Cuenta de GitHub creada
- ☐ Cuenta de Supabase creada
- ☐ Cuenta de Vercel creada (con GitHub)

3. Paso 1: Crear proyecto en Supabase

1 Ir a Supabase y crear proyecto

1. Ve a **supabase.com** e inicia sesión
2. Haz clic en **"New Project"**
3. Completa el formulario:
 - **Name:** app-5s (o el nombre que prefieras)
 - **Database Password:** Elige una contraseña segura y guárdala
 - **Region:** Selecciona la más cercana a tu ubicación
4. Haz clic en **"Create new project"**
5. Espera ~2 minutos mientras se crea el proyecto

4. Paso 2: Configurar base de datos PostgreSQL

2 Obtener las URLs de conexión

1. En tu proyecto Supabase, ve a **Settings** → **Database**
2. Busca la sección **"Connection string"**
3. Copia la URL con **"Connection pooling"** (puerto 6543) — esta es tu DATABASE_URL
4. Copia la URL **"Direct connection"** (puerto 5432) — esta es tu DIRECT_URL
5. Reemplaza [YOUR-PASSWORD] con la contraseña que elegiste

```
# Ejemplo de URLs (las tuyas serán diferentes):  
DATABASE_URL="postgresql://postgres.xxxx:PASSWORD@aws-0-eu-central-1.pooler.supabase.com:6543/postgres"  
DIRECT_URL="postgresql://postgres.xxxx:PASSWORD@aws-0-eu-central-1.supabase.com:5432/postgres"
```

Contraseña

Si tu contraseña contiene caracteres especiales como @, #, %, debes codificarlos como URL (ej: @ → %40). Puedes usar [urlencoder.org](https://www.urlencoder.org).

5. Paso 3: Crear bucket de almacenamiento

1. En tu proyecto Supabase, ve a **Storage** en el menú lateral
2. Haz clic en **"New bucket"**
3. Nombre: **photos**
4. Marca la opción **"Public bucket"** para que las fotos sean accesibles
5. Haz clic en **"Create bucket"**

Ahora obtén las credenciales de API:

1. Ve a **Settings** → **API**
2. Copia el **Project URL** — este es tu NEXT_PUBLIC_SUPABASE_URL
3. Copia el **Service role key** (secret) — este es tu SUPABASE_SERVICE_ROLE_KEY

Ejemplo:

```
NEXT_PUBLIC_SUPABASE_URL="https://xxxx.supabase.co"
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIs..."
```

Seguridad

NUNCA compartas el Service Role Key. Es la clave maestra de tu Supabase. Solo se usa en el servidor (backend), nunca en el navegador.

6. Paso 4: Subir código a GitHub

4

Crear repositorio y subir el código

1. Ve a github.com y haz clic en "New repository"
2. Nombre: **app-5s**
3. Marca "**Private**" (recomendado para apps de empresa)
4. No marques "Add a README"
5. Haz clic en "**Create repository**"

Ahora, en tu computadora, abre una terminal y ejecuta:

```
# Ir al directorio del proyecto
cd /home/z/my-project

# Inicializar git (si no existe)
git init

# Crear .gitignore
cat > .gitignore << 'EOF'
node_modules/
.next/
.env
.env.local
db/
public/uploads/
*.log
EOF

# Agregar todos los archivos
git add .

# Primer commit
git commit -m "App 5S lista para Vercel + Supabase"

# Conectar con GitHub (reemplaza TU-USUARIO)
git remote add origin https://github.com/TU-USUARIO/app-5s.git

# Subir el código
git push -u origin main
```

Autenticación GitHub

GitHub requiere un Personal Access Token (PAT) en vez de contraseña. Ve a GitHub → Settings → Developer settings → Personal access tokens → Genera uno con permisos de "repo".

7. Paso 5: Desplegar en Vercel

5

Conectar repositorio y desplegar

1. Ve a **vercel.com** e inicia sesión con GitHub
2. Haz clic en **"Add New..."** → **"Project"**
3. Selecciona tu repositorio **app-5s**
4. En "Configure Project", déjalo todo por defecto EXCEPTO:
 - **Framework Preset:** Next.js (se detecta automáticamente)
 - **Build Command:** `npx prisma generate && next build`
5. **NO desplegar todavía** — primero necesitamos las variables de entorno (Paso 6)

Build Command importante

El Build Command debe ser: `npx prisma generate && next build`. Esto genera el cliente de Prisma antes de compilar la app. Sin esto, el build fallará.

8. Paso 6: Configurar variables de entorno

6

Agregar variables de entorno en Vercel

1. En la pantalla de configuración del proyecto en Vercel, busca **"Environment Variables"**
2. Agrega las 4 variables siguientes:

Nombre	Valor	Obtener de
DATABASE_URL	postgresql://postgres...pooler...6543/postgres	Supabase → Settings → Database (pooling)
DIRECT_URL	postgresql://postgres...5432/postgres	Supabase → Settings → Database (direct)
NEXT_PUBLIC_SUPABASE_URL	https://xxxx.supabase.co	Supabase → Settings → API
SUPABASE_SERVICE_ROLE_KEY	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...	Supabase → Settings → API (service_role)

Ejemplo completo de las 4 variables:

```
DATABASE_URL="postgresql://postgres.xxxx:TU-PASSWORD@aws-0-eu-central-1.pooler.supabase.com:6543/postgres"
DIRECT_URL="postgresql://postgres.xxxx:TU-PASSWORD@aws-0-eu-central-1.supabase.com:5432/postgres"
NEXT_PUBLIC_SUPABASE_URL="https://xxxx.supabase.co"
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Después de agregar las variables, haz clic en **"Deploy"**.

9. Paso 7: Ejecutar migración de base de datos

7

Crear las tablas en Supabase

Después del primer despliegue (que puede fallar porque no hay tablas), necesitas crear las tablas en Supabase:

1. En tu proyecto Supabase, ve a **SQL Editor**
2. Haz clic en **"New query"**
3. Ejecuta la migración generada por Prisma. Para obtener el SQL:

```
# En tu terminal local (con DATABASE_URL configurado):  
npx prisma migrate diff \  
  --from-empty \  
  --to-schema-datamodel prisma/schema.prisma \  
  --script > migration.sql  
  
# Luego copia el contenido de migration.sql  
# y pégalo en el SQL Editor de Supabase
```

Alternativamente, si tienes `prisma-cli` configurado localmente con la URL de Supabase:

```
# Configura temporalmente la URL de Supabase  
export DATABASE_URL="postgresql://postgres:xxxx:PASSWORD@aws-0-eu-central-1.  
pooler.supabase.com:6543/postgres"  
export DIRECT_URL="postgresql://postgres:xxxx:PASSWORD@aws-0-eu-central-1.su  
pabase.com:5432/postgres"  
  
# Ejecuta la migración  
npx prisma db push
```

Después de crear las tablas, **re-despliega en Vercel** (Settings → Deployments → Redeploy).

Tu app estará lista

Después de la migración y el redeploy, tu app 5S estará accesible permanentemente en una URL como: `app-5s.vercel.app`

10. Flujo de trabajo diario

Una vez desplegada, el flujo para hacer mejoras es simple:

Ciclo de desarrollo

 Mejoras en Z →  git commit →  git push →  Vercel actualiza automáticamente

Comandos habituales

```
# 1. Haces cambios en tu app (en Z)

# 2. Guardas los cambios
git add .
git commit -m "Descripción del cambio"

# 3. Subes a GitHub (Vercel detecta y despliega automáticamente)
git push
```

En ~1-2 minutos, Vercel habrá desplegado la nueva versión. Tus usuarios verán la actualización automáticamente.

11. Solución de problemas

Problema	Solución
Build falla en Vercel	Verifica que el Build Command sea: <code>npx prisma generate && next build</code>
Error de conexión a BD	Verifica <code>DATABASE_URL</code> y <code>DIRECT_URL</code> en Vercel → Settings → Environment Variables
Las fotos no se suben	Verifica <code>NEXT_PUBLIC_SUPABASE_URL</code> y <code>SUPABASE_SERVICE_ROLE_KEY</code> . Verifica que el bucket "photos" sea público.
Página en blanco	Verifica que las tablas se crearon correctamente en Supabase → Table Editor
Error 500 en API	Revisa los logs en Vercel → Deployments → Function Logs
Los datos no se guardan	Verifica que <code>DIRECT_URL</code> use el puerto 5432 (no 6543) para migraciones

Supabase: 500MB de base de datos + 1GB de almacenamiento de fotos. Con fotos comprimidas (~100KB cada una), puedes almacenar ~10,000 fotos. Vercel: ancho de banda ilimitado para proyectos hobby.

¡Tu app 5S estará permanentemente en línea!

Sin costos, sin caducidad, sin servidor que mantener